

expense app — Worker Money: Settlement, Ledger, Advances, Payment

Dual-register guide: for developers AND for the owner learning the system. Canonical truth: [docs/ARCHITECTURE V2.md §11](#) + [ADR-0005](#) / [0007](#) / [0009](#). First-read: [docs/PROJECT KNOWLEDGE MAP.md](#). (Pre-V2 payroll docs are archived under [docs/archive/production/.](#))

Purpose

Yeh app **paisa** sambhalta hai — aur sirf paisa. Workers ka kaam kitna hua yeh **production** app ka sach hai; **us kaam ke paise kab bane, kitne bane, kitne diye** — yeh sab yahan hai.

Business responsibility

One sentence: **settlement banata hai earning; payment deta hai cash; ledger sab kuch yaad rakhta hai — hamesha ke liye.**

- An **AddaSettlement** is the owner's approval event: "is Adda ka kaam check ho gaya, ab paise book karo." (Settlement $\neq$ payment!)
- A **PayrollSettlement** is now PAYMENT-ONLY: cash diya, bas.
- A **WorkerAdvance** is a loan pool — recovery happens ONLY at Adda settlement, owner ki marzi se, per-advance.
- The **WorkerLedgerEntry** table is the financial truth: append-only, kabhi edit nahi, kabhi delete nahi.

What tables are created (database impact)

Table	One row means	Lifecycle
<code>expense_workerledgerentry</code>	one money movement (credit/debit) for one worker	INSERT-only, forever. Mistake = opposite-direction REVERSAL row, never UPDATE
<code>expense_addasettlement</code>	one settlement EVENT on one Adda (<code>ADST-0007</code>)	draft → finalized → reversed/superseded; drafts may be discarded (no money yet)
<code>expense_addasettlementitem</code>	FROZEN per-worker snapshot of what the owner approved	write-once at finalize; NEVER recomputed (audit record)
<code>expense_stageworkassignment</code> (SWA)	one EARNING LINE (worker × stage × color/size)	created at settlement finalize (era-B) or, historically, at allocation (era-A — <code>adda_settlement</code> NULL); corrected by soft <code>voided_at</code> , never deleted
<code>expense_payrollsettlement</code>	one CASH PAYMENT to one worker (<code>SETL-0012</code>)	append-only
<code>expense_payrollsettlementitem</code>	one advance-recovery line (XOR-parented to payment OR adda-settlement)	append-only + <code>reversed_at</code> stamp (never signed/edited)
<code>expense_workeradvance</code>	one loan given to a worker	append-only; outstanding = live SUM minus recoveries
<code>expense_workerprofile</code>	bank/UPI/opening-balance per worker	normal mutable row (not history)

Example row (ledger): `id=10, worker=utest, entry_type=credit, category=stage_earning, amount=180.00, assignment=SWA#42, notes='ADST-0003 · 3-PATTI-001'` — matlab: “utest ko 3-PATTI-001 ke settlement ADST-0003 se ₹180 ka earning मिला, line SWA#42 se.”

How data flows through this app

```

production truth (WSC: reported/verified qty, frozen expected_rate)
  | (read-only – settlement NEVER edits production)
  ▼
AddaSettlement DRAFT —discard→ (deleted; no money existed)
  | finalize (adda_settlement_service – THE chokepoint)
  ▼
per line: SWA earning line + ledger CREDIT (stage_earning)
per advance chosen: ledger DEBIT (advance_recovery) + PSI row
per worker: FROZEN AddaSettlementItem + settlement totals
  |
  ▼ mistake found?
reverse_adda_settlement → compensating ledger rows + SWA voided
                        → optional successor draft (supersede chain)
  |
  ▼ cash day
PayrollSettlement (payment-only) → ledger DEBIT (settlement_payment)

```

Worker visibility ladder (V2-3): **Expected (unsettled)** — frozen `expected_*` on WSC, sirf dikhane ke liye → **Earned (settled)** — ledger → **Paid (cash)**. Teeno kabhi overlap nahi karte (tested).

What services are allowed to write (single-writer rules)

Table	SOLE writer	Enforced by
WorkerLedgerEntry	<code>ledger_service</code>	CLAUDE rule 5 + code review
AddaSettlement + Item	<code>adda_settlement_service</code>	CI gate [4b/4]
StageWorkAssignment	<code>allocation_service</code> (era-A, lever-only) + <code>adda_settlement_service</code> (era-B)	CI gate [4c/4]
PayrollSettlement(+Item)	<code>settlement_service</code> (payment-only since V2-2)	review
WorkerAdvance	<code>payroll_service.record_advance</code>	review

WHY THIS DESIGN EXISTS / WHAT BREAKS IF BYPASSED

- **Why one writer per money table?** Paise ka hisaab tab hi bharosemand hai jab likhne ka EK hi darwaza ho. Har guard (double-credit, recovery ≤ remaining, lock order) us darwaze pe baitha hai.

- **What breaks if you write the ledger directly?** Balances are live SUMs — a stray row silently changes what the factory owes a worker, with no reversal path and no settlement reference. The append-only promise means your mistake is PERMANENT history.
- **What breaks if you edit a frozen `AddaSettlementItem`?** The audit record of “what the owner approved” stops matching the ledger — the entire correction system (reverse/supersede) assumes snapshots never move.
- **What breaks if settlement money is touched outside the lifecycle?** Exactly the two V2-3 PR-A holes: reopen/void used to reverse settlement credits while the settlement still read FINALIZED. Now both refuse and name the ADST reference.

Chokepoint services (who calls / what writes / invariant protected)

`ADDA_SETTLEMENT_SERVICE` — THE MONEY EVENT

- **Who calls:** management UI (Payroll → Adda Settlements) only; `_ensure_management` inside.
- **Writes:** SWA lines, ledger credits/debits (via `ledger_service`), PSI recovery rows, frozen items, settlement status, `AddaHistory` events.
- **Why single writer:** finalize is a multi-table money transaction with a strict lock order (advisory lock 5374 → ADST row → stage records → worker profiles → advances). Two writers = deadlocks + double credits.
- **Invariant protected:** *a contribution line is paid at most once, ever* (cross-era guard both directions), and *every paise is traceable*: item ↔ SWA ↔ ledger ↔ WSC.settlement_line.

`LEDGER_SERVICE` — THE PEN THAT WRITES MONEY

- **Who calls:** other expense services only — never views.
- **Writes:** `WorkerLedgerEntry` INSERTs (`log_credit` / `log_debit` / `reverse_entry`).
- **Invariant:** append-only; a reversal copies `entry_date` so monthly totals net correctly in-period.

`ALLOCATION_SERVICE` — THE LEGACY PATH (ROLLBACK LEVER ONLY)

- Since V2-3 the default refuses (`LEDGER_CREDIT_AT_ALLOCATION=False`); env True restores it fully (tested by 8 pinned test classes).
- `void_allocation` corrects era-A lines only; era-B refuses → “reverse the settlement instead.”

SETTLEMENT_SERVICE — PAYMENT-ONLY

- Rejects any recovery input loudly (recovery moved to Adda settlement).

PAYROLL_SERVICE — THE READ LAYER

- All dashboards/My Earnings read through here. Knows the era rules so templates never do.

Views (what enters)

`/expense/my/` worker self-view (never role-gated, always self-scoped) · `/expense/payroll/`
 + `/workers/<id>/` management · `/workers/<id>/settle/` CASH payment ·
`/advances/add/` · `/expense/settlements/...` the settlement queue/detail/actions. All money
 POSTs: management-gated + confirm dialogs.

Common mistakes (developers: DO NOT)

1. Never `WorkerLedgerEntry.objects.create(...)` outside `ledger_service`.
2. Never UPDATE/DELETE money rows — reversal/void/stamp patterns only.
3. Never read `WSC.expected_*` as money (visibility only — ADR-0005).
4. Never sum `processing_cost + settled labor` (same labor twice — ADR-0009).
5. Per-Adda actual labor = Σ non-voided SWA snapshots (both eras) — never Σ settlement totals (partial/reversed chains mis-sum).
6. Don't add fields to frozen tables casually — frozen means frozen.

Django Learning Notes (is app mein kaunse patterns kyun)

- `transaction.atomic` + `select_for_update()` : finalize/reverse pura ek transaction hai; `select_for_update` row ko lock karta hai taaki do managers ek saath finalize na kar dein. Lock ORDER fixed hai (5374 → ADST → SR → profile → advance) kyunki alag order = deadlock.
- `pg_advisory_xact_lock(5374)` : Postgres ka app-level lock — reference numbering (ADST-0007) race-safe banata hai bina table banaye.
- `on_delete=PROTECT` on every money FK: ledger row ke peeche ka Adda/ worker/SWA kabhi delete nahi ho sakta — history anchor hai. (CASCADE sirf wahan jahan child bina parent ke meaningless ho; SET_NULL sirf optional references pe.)
- **CheckConstraints** (DB-level): `finalized ⇒ settled_at`, `recovered ≤ outstanding`, XOR exactly-one-parent on PSI — app bug bhi DB ko jhooth nahi likhwa sakta.

- **No signals** (ADR-0001): sab kuch service functions mein, explicitly — “yeh kab chala?” ka jawab hamesha stack trace mein milta hai.
- **Frozen snapshot vs live SUM**: snapshot = “us waqt kya approve hua” (kabhi recompute nahi); balance = “abhi kitna baaki” (hamesha SUM). Dono ka mix hi audit + accuracy dono deta hai.

Related ADRs

0001 (services own writes) · 0002 (single writer) · 0005 (Option B — truth split) · 0007 (era cu-tover + lever) · 0009 (cost truth / never-add) · 0010 §4 (rework case-scoping).

Real factory example (end-to-end)

3-PATTI-001: utest ne cutting pe 60 Red Size-1 + 15 Size-2 report kiya (15 verified ho ke 15 hi rahe). Stage complete → settlement queue mein Adda “Ready ₹225”. Accountant draft kholta hai → lines check → variance “73 packed, 2 missing” → Finalize → ledger mein do credit (₹180 + ₹45), frozen item ₹225, ADST-0003. Galti mili? Reverse & settle again → compensating rows, naya draft, chain ADST-0001 → 0002 → 0003 hamesha dikhega. Phir cash day pe worker detail → Pay → ₹225 — payment-only screen, ledger DEBIT, ho gaya.